

LAPORAN MILESTONE 1 TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA
Lexical Analysis



Disusun oleh:

Juan Oloando Simanungkalit	13524032
Edward David Rumahorbo	13524036
Reinhard Alfonzo Hutabarat	13524056
Manuel Timothy Silalahi	13524102

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL.GANESA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi.....	2
BAB 1	
LANDASAN TEORI.....	3
1.1. Analisis Leksikal (Lexical Analysis).....	3
1.2. Token.....	3
1.3. Deterministic Finite Automata (DFA).....	4
1.4. Arsitektur dan Data Flow pada Analisis Leksikal.....	4
BAB 2	
PERANCANGAN DAN IMPLEMENTASI.....	6
2.1. Perancangan.....	6
2.1.1 DFA Literals.....	8
2.1.2 DFA Keywords.....	10
2.1.3 DFA Operators.....	12
2.1.4 DFA Comment & Symbol.....	13
2.2. Implementasi.....	14
2.2.1. Kakas yang Digunakan.....	14
2.2.2. Struktur Program.....	14
2.2.3. Fungsi dan Kelas Utama.....	15
2.2.4. Justifikasi Implementasi.....	22
BAB 3	
PENGUJIAN.....	22
3.1. Test Case 1.....	22
3.2. Test Case 2.....	23
3.3. Test Case 3.....	26
3.4. Test Case 4.....	26
3.5. Test Case 5.....	27
BAB 4	
KESIMPULAN DAN SARAN.....	28
4.1 Kesimpulan.....	28
4.2 Saran.....	29
LAMPIRAN.....	29
REFERENSI.....	30

BAB 1

LANDASAN TEORI

1.1. Analisis Leksikal (*Lexical Analysis*)

Analisis leksikal merupakan fase pertama dalam tahapan kompilasi sebuah *compiler*. Tugas utama dari *lexical analyzer* adalah membaca karakter-karakter input secara mentah dari program sumber, mengelompokkannya, dan menghasilkan urutan *token* sebagai keluaran untuk dikirim ke tahapan *parser*. Dalam menjalankan proses komputasi ini, terdapat perbedaan antara *token*, *pattern*, dan *lexeme*. *Token* adalah pasangan yang terdiri dari nama *token* dan nilai atribut opsional yang merepresentasikan jenis unit leksikal tertentu di dalam bahasa pemrograman. *Pattern* merupakan deskripsi dari kaidah atau bentuk yang harus dipenuhi oleh suatu *lexeme* untuk dapat dikategorikan ke dalam suatu *token*. Sedangkan *lexeme* adalah urutan karakter aktual dalam program sumber yang cocok dengan *pattern* dari suatu *token* tertentu dan diidentifikasi oleh *lexical analyzer* sebagai instansi nyata dari *token* tersebut.

Selain mengidentifikasi *lexeme*, *lexical analyzer* juga bertugas membersihkan input dengan membuang karakter-karakter yang tidak relevan dengan logika program. Karakter-karakter seperti *whitespace* serta komentar akan dihapus dari teks sumber agar tidak membebani tahapan kompilasi selanjutnya. Proses pemindaian ini melakukan kompresi karakter *whitespace* yang berurutan menjadi satu kesatuan dan mengeliminasi teks komentar, sehingga *lexical analyzer* dapat mengubah sisa karakter yang relevan menjadi *token*.

1.2. *Token*

Dalam konteks bahasa pemrograman dan *compiler*, *token* merupakan *lexical unit* fundamental yang diproses oleh *parser* sebagai representasi dari elemen teks sumber. Secara struktural, sebuah *token* direpresentasikan sebagai pasangan yang terdiri dari dua komponen utama, yaitu *token name* dan *attribute value*. *Token name* adalah simbol abstrak yang mendeskripsikan kategori atau jenis *lexical unit*, seperti penanda untuk sebuah kata *keyword* atau *identifier*. Sementara itu, nilai atribut digunakan untuk menyimpan informasi spesifik mengenai *lexeme* aktual yang ditemukan, terutama ketika suatu pola dapat menghasilkan lebih dari satu kemungkinan *lexeme*. *Lexeme* itu sendiri adalah urutan karakter nyata di dalam program sumber yang cocok dengan pola dari suatu *token* dan diidentifikasi secara sah sebagai instansi dari *token* tersebut.

Sebagian besar bahasa pemrograman mengklasifikasikan unit leksikalnya ke dalam beberapa kategori *token* umum, yang meliputi *token* kata kunci yang polanya adalah karakter kata kunci itu sendiri, *identifier token*, *token* konstanta seperti angka atau literal string, *token* operator, serta *token* tanda baca seperti tanda kurung atau titik koma. Anatomi atribut untuk setiap kategori ini memiliki karakteristik yang berbeda-beda.

Token seperti kata kunci, tanda baca, dan beberapa operator sering kali tidak memerlukan nilai atribut tambahan karena *token name* saja sudah cukup untuk mengidentifikasi maknanya secara tunggal. Sebaliknya, *token* yang merepresentasikan *identifier* biasanya memiliki nilai atribut berupa *pointer* yang merujuk pada entri di dalam tabel simbol untuk menyimpan informasi rinci tentang *identifier* tersebut. Demikian pula, *token* konstanta numerik umumnya wajib menyertakan atribut berupa nilai integer aktual atau *pointer* ke *string* dari angka tersebut agar terjemahan nilainya tidak hilang saat proses kompilasi berlanjut.

1.3. Deterministic Finite Automata (DFA)

Deterministic Finite Automata (DFA) adalah sebuah model komputasi matematis berupa mesin otomata berhingga yang bertumpu pada lima komponen utama, yaitu himpunan keadaan (*states*), himpunan simbol input, serta sebuah fungsi transisi. Secara operasional, mesin ini memiliki tepat satu keadaan awal yang merepresentasikan kondisi sistem sebelum memproses input, serta himpunan keadaan akhir yang menandakan bahwa suatu urutan input merupakan string yang sah. Sifat deterministik pada DFA menunjukkan bahwa untuk setiap keadaan saat ini dan setiap simbol input spesifik yang diberikan, mesin hanya memiliki tepat satu kemungkinan keadaan tujuan untuk melakukan transisi. Kepastian langkah tunggal ini membedakannya secara fundamental dari otomata nondeterministik yang dapat bercabang ke dalam beberapa keadaan sekaligus secara bersamaan (misalnya NFA).

DFA diimplementasikan sebagai mesin fundamental pendukung *lexical analysis* untuk merepresentasikan *pattern* dan mengenali *token* secara otomatis. Pola-pola *lexeme* yang awalnya didefinisikan menggunakan *regular expression* dapat ditransformasikan ke dalam bentuk otomata deterministik ini. DFA mensimulasikan proses pengenalan dengan membaca string karakter input secara terstruktur dan berpindah dari satu *state* ke *state* lain melalui fungsi transisinya. Keberhasilan mesin mencapai *accepting state* menandakan bahwa sebuah pola *lexeme* telah dikenali dengan valid. Melalui kepastian transisi status ini, DFA menyediakan sebuah algoritma konkret dan sangat efisien untuk mengevaluasi setiap pembacaan karakter dari program sumber hingga membentuk unit *token* yang utuh.

1.4. Arsitektur dan Data Flow pada Analisis Leksikal

Lexical analysis atau *scanning* merupakan fase pertama dalam arsitektur *compiler*. Dalam pemodelan *data flow*-nya, mesin *lexical analyzer* bertindak sebagai pintu gerbang utama yang menerima masukan secara langsung berupa *character stream* dari program sumber. Proses transformasi internal kemudian terjadi ketika mesin membaca aliran karakter tersebut dan mengelompokkannya menjadi sekumpulan urutan karakter yang bermakna logis, yang disebut sebagai *lexeme*. Pemrosesan awal ini memegang peran krusial dalam

menyaring teks mentah menjadi unit-unit leksikal yang lebih terstruktur dan ringkas sebelum dievaluasi lebih lanjut.

Sebagai hasil akhir dari transformasi tersebut, *lexical analyzer* memproduksi output berupa *token stream*. Untuk setiap *lexeme* yang ditemukan, mesin menghasilkan sebuah *token* dalam format pasangan yang memuat nama *token* sebagai simbol abstrak serta nilai atribut opsionalnya. Aliran *token* yang telah dikonstruksi ini kemudian diteruskan secara utuh ke tahap *syntax analyzer* atau *parser* untuk memandu proses pembentukan struktur tata bahasa program. Secara implementasi operasional, aliran data ini umumnya terjadi melalui skema interaksi dinamis dua arah. *Parser* memanggil *lexical analyzer* untuk membaca karakter masukan secara terus-menerus hingga sebuah *lexeme* baru berhasil diidentifikasi, lalu *token* yang merepresentasikannya dikembalikan ke *parser* untuk dianalisis.

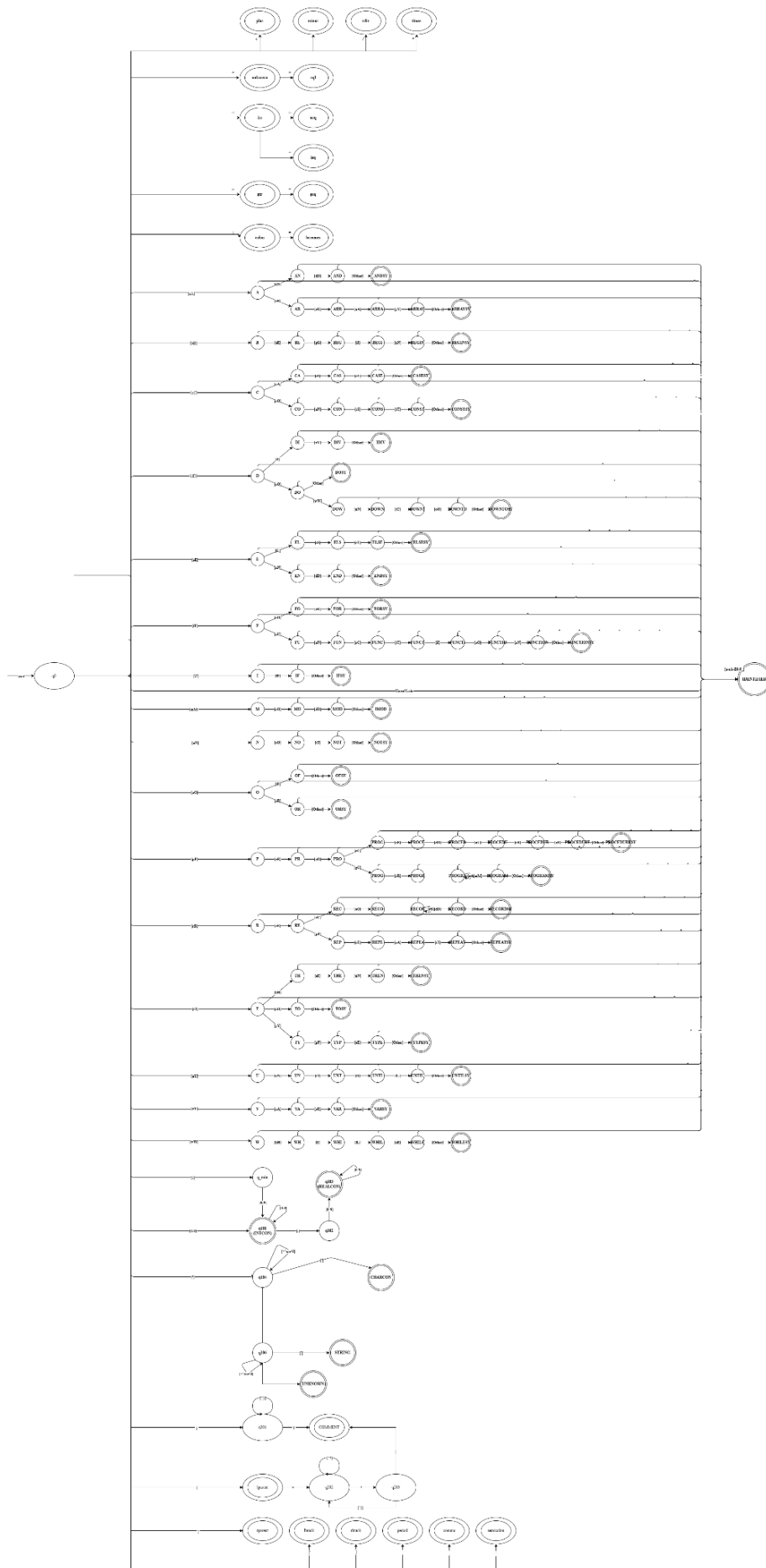
BAB 2

PERANCANGAN DAN IMPLEMENTASI

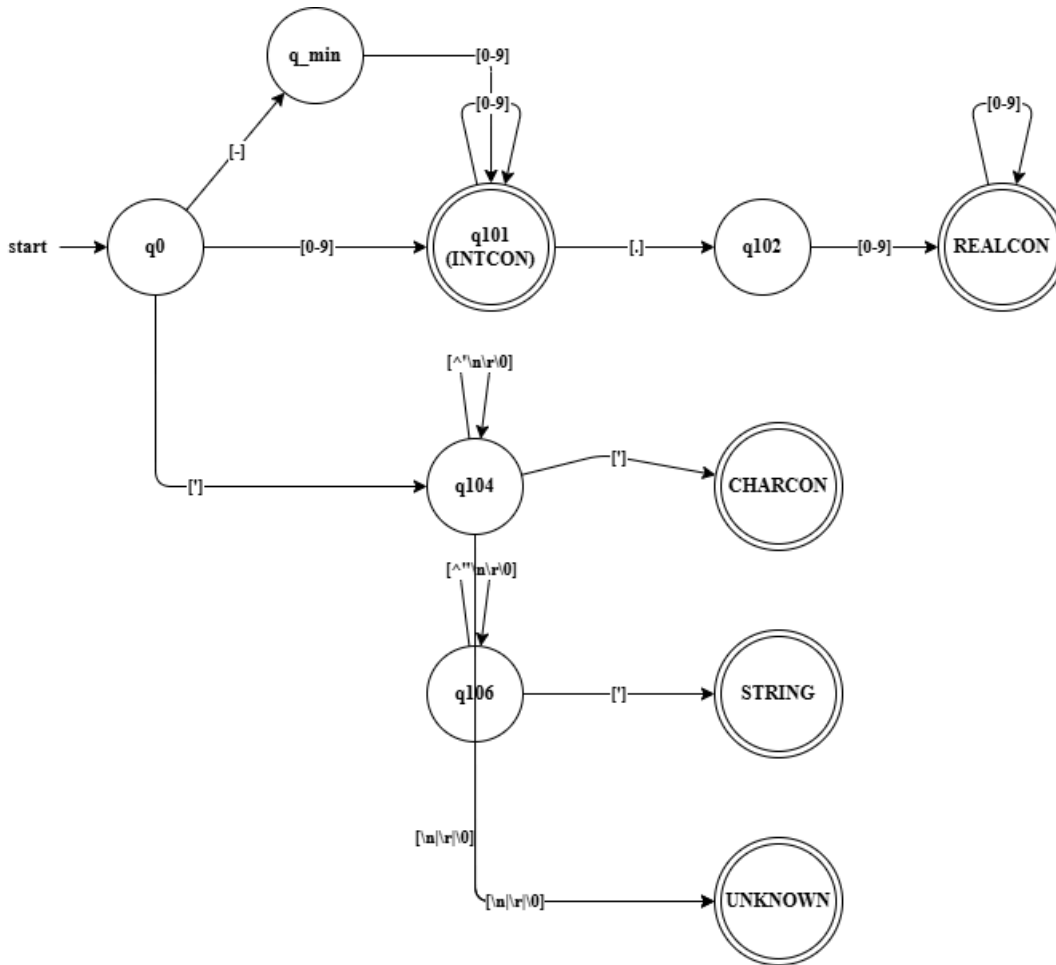
2.1. Perancangan

Dalam pembuatan lexical analyzer, kami menggunakan diagram DFA untuk menggambarkan state dan transisi dari mesin DFA. Berikut merupakan diagram DFA dan keterangan state:

*Untuk gambar yang lebih jelas silahkan akses link berikut:
<https://drive.google.com/file/d/1JQkafdOFwvhX3yxJjynyNv-1ERKnLzq8/view?usp=sharing>



2.1.1 DFA Literals

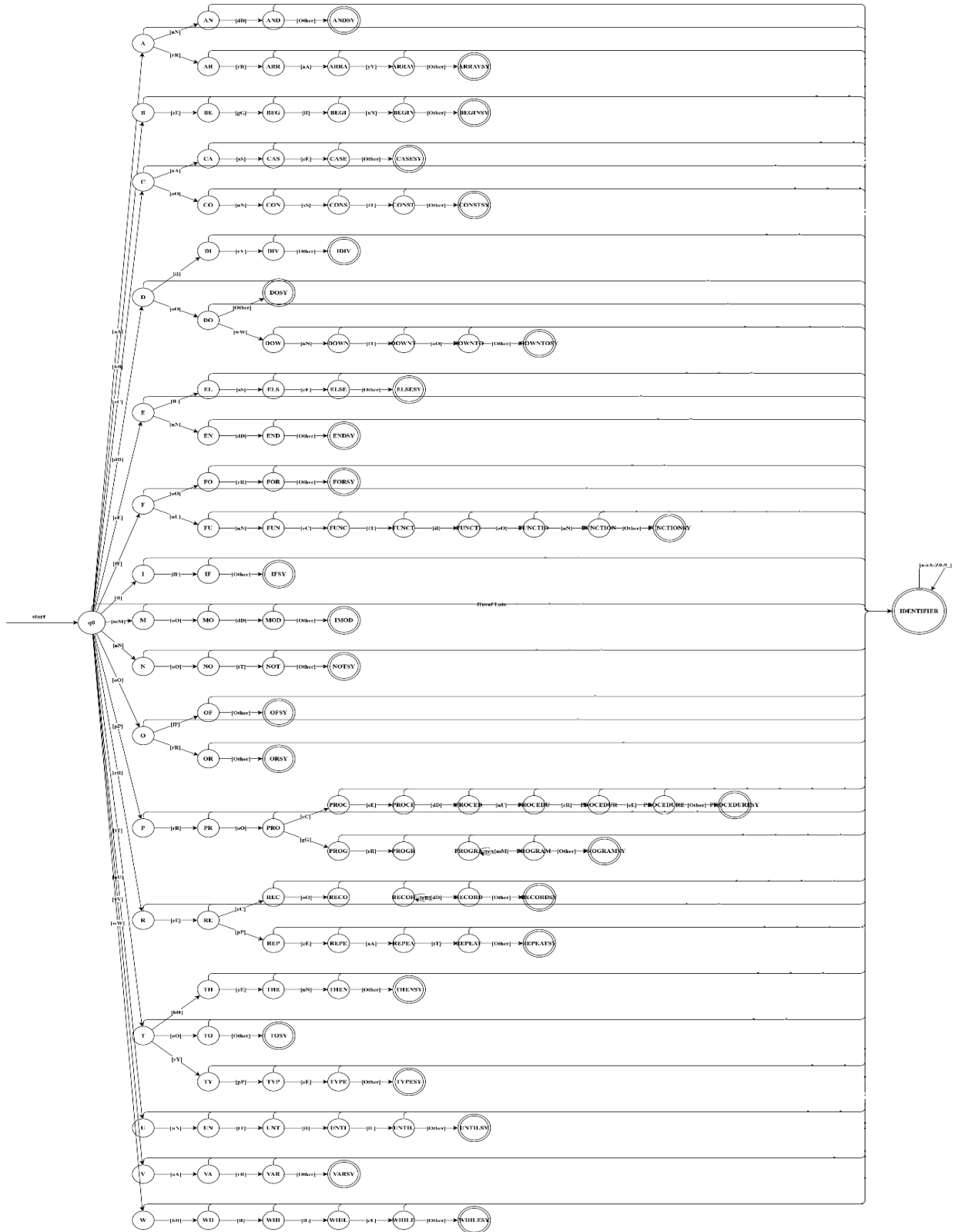


Tabel Keterangan State

q0	Start state (DFA belum membaca apa pun)
q_min	DFA membaca simbol -
intcon	DFA membaca satu atau lebih angka (0..9)
q102	DFA membaca . dan tepat sebelumnya membaca angka apa pun
realcon	DFA membaca satu atau lebih angka (0..9) setelah membaca .
q104	DFA membaca ' sebagai karakter pertama atau karakter apapun selain '

	\n \r \0 setelah membaca ‘
charcon	DFA membaca ‘ setelah membaca satu karakter apapun selain ‘ dan membaca karakter ‘ sebagai karakter awal
q106	DFA membaca dua atau lebih karakter apapun selain ‘ \r \n \0 setelah membaca simbol ‘ di awal
string	DFA membaca ‘ setelah membaca dua atau lebih karakter apapun selain ‘ dan membaca karakter ‘ sebagai karakter awal

2.1.2 DFA Keywords

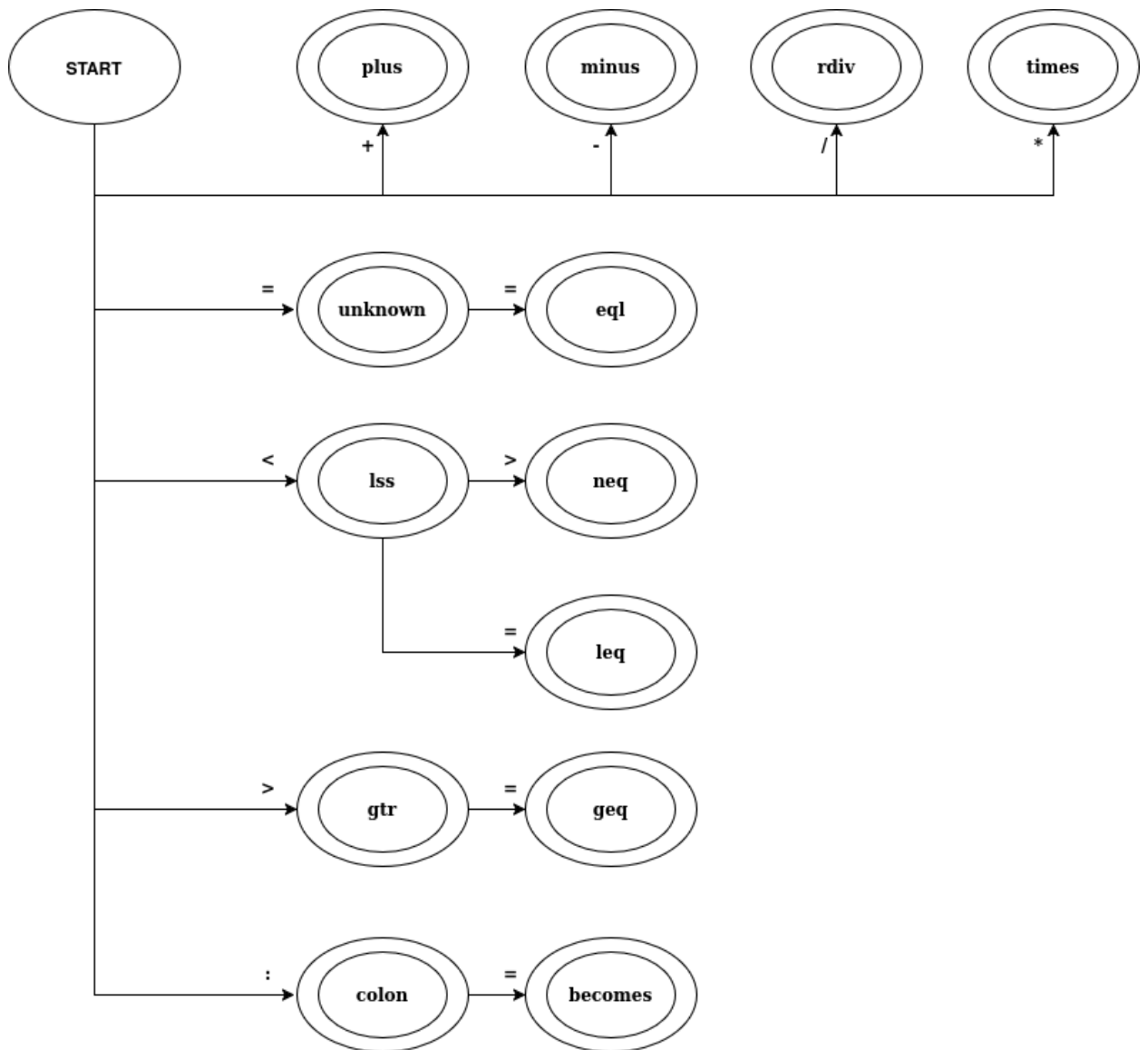


Tabel Keterangan State

q0	Start State
State Perantara (q_i, q_en, q_prog, dll.)	DFA sedang dalam proses membaca dan merangkai karakter demi karakter. Pada fase ini, teks belum membentuk kata kunci yang utuh atau sedang diuji kecocokannya dengan pola <i>keyword</i> .
State Operator Teks (q_and, q_or)	State Perantara Khusus, DFA sedang membaca rangkaian karakter yang berpotensi menjadi operator berbasis teks, seperti div, mod, and, or, not.
State Final Keyword (BEGINSY, IFSY, PROGRAMSY, dll.)	DFA telah berhasil mencocokkan input dengan tepat sesuai salah satu kata kunci Arion dan bertemu dengan karakter pemisah (<i>delimiter</i>). Token <i>keyword</i> spesifik dikembalikan.
IDENTIFIER (Final State)	DFA menerima rangkaian huruf dan angka yang tidak cocok dengan kata kunci manapun pada saat pemutusan jalur, sehingga diklasifikasikan sebagai nama variabel/konstanta biasa.

**Note: Diagram DFA di atas merepresentasikan logika automata murni (Prefix-Tree / Trie) secara teoritis, di mana pemisahan setiap kata kunci dilakukan secara hardcode huruf demi huruf melalui ratusan state perantara. Namun, pada implementasi kode kami, arsitektur ini dioptimalkan. Pemindai (lexer) akan membaca seluruh urutan huruf dan angka sebagai satu kesatuan Identifier tunggal melalui looping DFA sederhana. Setelah sebuah kata utuh terkumpul, program akan mencocokkannya menggunakan struktur data Lookup Table (Hash Map) untuk menentukan apakah kata tersebut adalah Keyword atau Identifier biasa. Modifikasi implementasi ini dilakukan untuk menjaga efisiensi kinerja kompilator, menghindari spaghetti code akibat hardcode percabangan yang masif, serta memastikan kemudahan pemeliharaan (maintainability) sistem di masa depan.*

2.1.3 DFA Operators

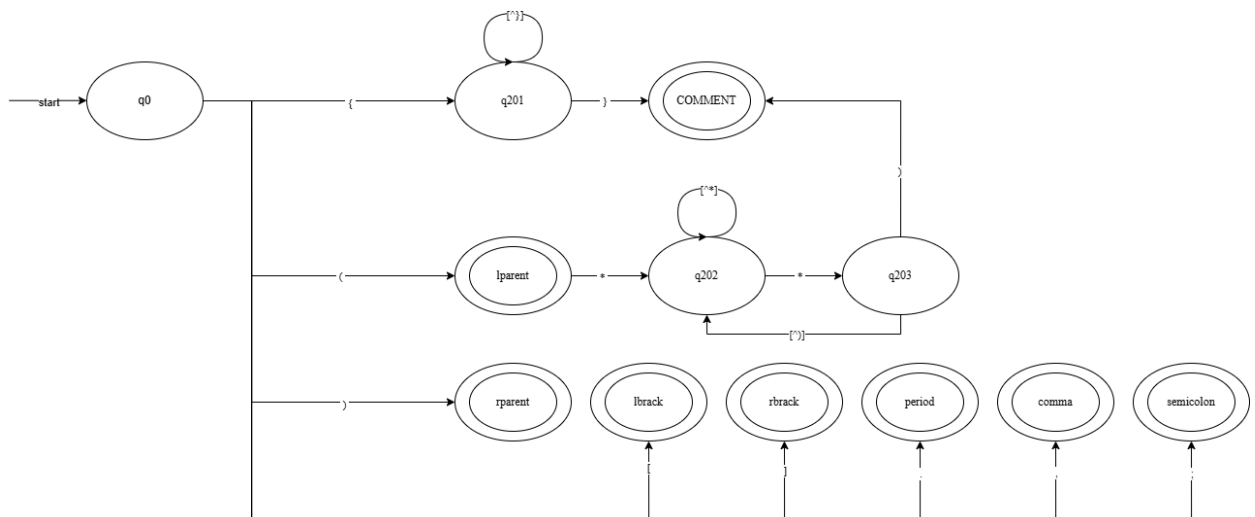


Tabel Keterangan State

q0	Start state (DFA belum membaca apapun)
plus	DFA membaca simbol +
minus	DFA membaca simbol -
rdiv	DFA membaca simbol /
times	DFA membaca simbol *
eq	DFA membaca simbol = kedua secara

	berturut-turut
lss	DFA membaca simbol >
neq	DFA membaca simbol > dan tepat sebelumnya membaca simbol <
leq	DFA membaca simbol = dan tepat sebelumnya membaca simbol <
gtr	DFA membaca simbol >
geq	DFA membaca simbol = dan tepat sebelumnya membaca simbol >
colon	DFA membaca simbol :
becomes	DFA membaca simbol = dan tepat sebelumnya membaca simbol :

2.1.4 DFA Comment & Symbol



Tabel Keterangan State

q0	Start state (DFA belum membaca apapun)
q201	DFA membaca { atau DFA membaca karakter apapun selain }
lparent	DFA membaca simbol (

q202	DFA membaca * dan tepat sebelumnya membaca (atau DFA membaca karakter apapun selain * dan sebelumnya sudah pernah membaca *
q203	DFA membaca * dan karakter sebelumnya bukan (
comment	DFA membaca } dan sebelumnya telah membaca { atau DFA membaca) dan tepat sebelumnya membaca *
rparent	DFA membaca simbol)
lbrack	DFA membaca simbol [
rbrack	DFA membaca simbol]
period	DFA membaca simbol .
comma	DFA membaca simbol ,
;	DFA membaca simbol ;

2.2. Implementasi

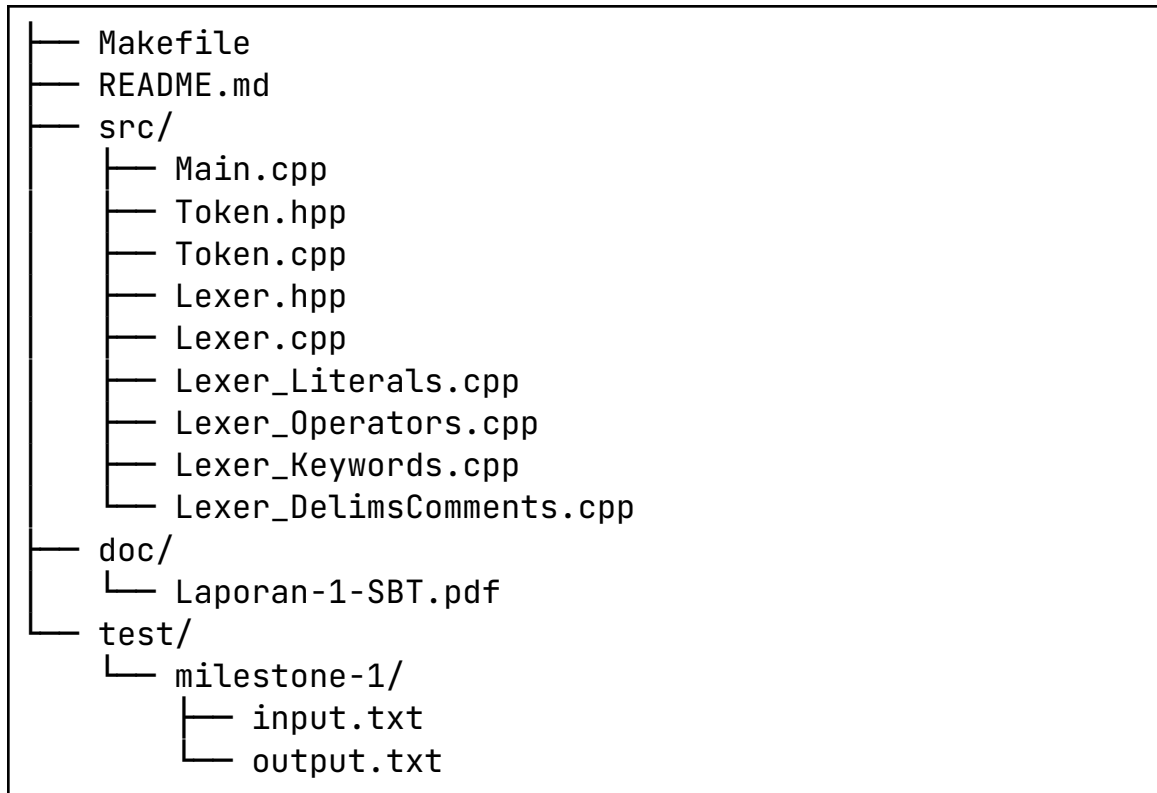
2.2.1. Kakas yang Digunakan

Bahasa utama yang digunakan dalam pembuatan Arion Lexical Analyzer adalah C++. Bahasa C++ dapat melakukan manipulasi data tingkat rendah (low-level) secara efisien, yang berperan dalam proses pembacaan karakter satu per satu dari file masukan. Untuk mentransformasi kode sumber menjadi program executable. Pada proses pengembangan Arion Compiler, digunakan GNU C++, yaitu g++.

2.2.2. Struktur Program

Berikut adalah struktur program *Lexical Analyzer* untuk bahasa pemrograman Arion:

SBT-Tubes-IF2224-2026/



2.2.3. Fungsi dan Kelas Utama

Dalam implementasi *Lexical Analyzer* ini, terdapat beberapa *class* dan fungsi utama yang menjadi bagian utama dari pemrosesan aliran karakter menjadi *token*:

1. *Class Token dan Enumerasi TokenType*

Komponen ini merupakan representasi struktur data fundamental dari hasil analisis leksikal. *TokenType* menggunakan enum class untuk mendefinisikan 52 jenis *token* yang dikenali oleh bahasa Arion secara ketat (*type-safe*). Sementara itu, *class Token* bertugas mengenkapsulasi jenis *token* tersebut bersama dengan *lexeme*, serta merekam metadata posisi berupa baris dan kolom untuk mendukung pelacakan *lexical error*.

```
enum class TokenType {  
    // Tipe Data & Konstanta  
    INTCON, REALCON, CHARCON, STRING,  
  
    // Operator Logika & Aritmatika  
    NOTSY, PLUS, MINUS, TIMES, IDIV, RDIV, IMOD, ANDSY,  
    ORSY,
```

```

    // Operator Relasional
    EQL, NEQ, GTR, GEQ, LSS, LEQ,

    // Simbol & Tanda Baca
    LPARENT, RPARENT, LBRACK, RBRACK, COMMA, SEMICOLON,
    PERIOD, COLON, BECOMES,

    // Keywords / Reserved Words
    CONSTSY, TYPESY, VARSY, FUNCTIONSY, PROCEDURESY,
    ARRAYSY, RECORDSY, PROGRAMSY,
    BEGINSY, IFSY, CASESY, REPEATSY, WHILESY, FORSY,
    ENDSY, ELSESY, UNTILSY, OFSY,
    DOSY, TOSY, DOWNTOSY, THENSY,

    // Identifier & Lain-lain
    IDENT, COMMENT,

    // Penanda khusus untuk internal lexer
    END_OF_FILE, UNKNOWN
};

class Token {
public:
    TokenType type;
    std::string value; // Lexeme (teks asli dari source
code)
    int line;          // Opsional tapi sangat disarankan
untuk error handling
    int column;        // Opsional tapi sangat disarankan
untuk error handling

    // Constructor
    Token(TokenType type, std::string value, int line = 0,
int column = 0);

    // Fungsi untuk mengubah token menjadi string (berguna
untuk output file .txt)
    std::string toString() const;
};

```


2. *Class Lexer*

Class Lexer adalah representasi utama dari mesin DFA di dalam memori. Kelas ini mengelola *state* internal proses *tokenization*, melacak kursor pembacaan *currentChar*, baris, dan kolom. *Class* ini memiliki metode primitif pembantu seperti *advance()* untuk mengonsumsi karakter secara berurutan, dan *retract()* untuk membatalkan pembacaan dengan mengembalikan karakter terakhir ke dalam aliran input. Penggunaan metode *retract()* ini secara harfiah merepresentasikan final state bertanda bintang (*) pada teori DFA murni, yang sangat esensial untuk menangani karakter yang terlanjur dibaca namun ternyata bukan bagian dari *token* yang sedang dievaluasi.

```
class Lexer {
private:
    std::ifstream file;
    char currentChar;
    int currentLine;
    int currentColumn;

    void advance();
    void retract();
    void skipWhitespace();

    TokenType prevTokenType = TokenType::UNKNOWN;

    Token lexLiteral();

    Token lexOperator();

    Token lexIdentifierOrKeyword();

    Token lexDelimiterOrComment();

public:
    Lexer(const std::string& filename);

    ~Lexer();

    Token getNextToken();
```

```
std::vector<Token> tokenizeAll();
};
```

3. Fungsi `Lexer::getNextToken()`

Metode ini adalah *dispatcher* utama dari mesin DFA. Fungsi ini dieksekusi setiap kali *loop* utama meminta *token* berikutnya. Secara logis, fungsi ini mengevaluasi `currentChar` dan mengarahkan transisi *state* ke fungsi pembantu yang spesifik, seperti `lexNumber()` untuk literal angka, `lexStringOrChar()` untuk teks, `lexOperator()` untuk simbol aritmatika dan komparasi, serta `lexIdentifier()` untuk kata kunci. Fungsi ini memastikan pembacaan dilakukan menggunakan metode *longest match* dan deterministik, memanfaatkan `retract()` jika tebakan *state* menemui jalan buntu pada karakter berikutnya.

```
Token Lexer::getNextToken() {
    skipWhitespace();

    if (currentChar == '\0') {
        return Token(TokenType::END_OF_FILE, "",
currentLine, currentColumn);
    }

    Token result(TokenType::UNKNOWN, "", 0, 0);

    if (std::isalpha(static_cast<unsigned
char>(currentChar)) || currentChar == '_') {
        result = lexIdentifierOrKeyword();
    }
    else if (currentChar == '-') {
        bool isUnary = (prevTokenType != TokenType::IDENT
&&
                                prevTokenType != TokenType::INTCON
&&
                                prevTokenType !=
TokenType::REALCON &&
                                prevTokenType !=
TokenType::RPARENT &&
                                prevTokenType !=
```

```

TokenType::RBRACK);
    if (isUnary) {
        int saveLine = currentLine;
        int saveCol = currentColumn;
        char saveCh = currentChar;
        advance();
        if (std::isdigit(static_cast<unsigned
char>(currentChar))) {
            retract();
            currentChar = saveCh;
            currentLine = saveLine;
            currentColumn = saveCol;
            result = lexLiteral();
        } else {
            retract();
            currentChar = saveCh;
            currentLine = saveLine;
            currentColumn = saveCol;
            result = lexOperator();
        }
    } else {
        result = lexOperator();
    }
}
else if (currentChar == '+') {
    result = lexOperator();
}
else if (std::isdigit(static_cast<unsigned
char>(currentChar)) ||
        currentChar == '\\' || currentChar == '"') {
    result = lexLiteral();
}
else if (currentChar == '(' || currentChar == ')' ||
        currentChar == '[' || currentChar == ']' ||
        currentChar == ',' || currentChar == ';' ||
        currentChar == '.' || currentChar == '{') {
    result = lexDelimiterOrComment();
}
else if (currentChar == '*' || currentChar == '/' ||
        currentChar == '=' || currentChar == '<' ||

```

```

        currentChar = '>' || currentChar = ':') {
            result = lexOperator();
        }
        else {
            int errLine = currentLine;
            int errCol = currentColumn;
            char bad = currentChar;
            advance();
            result = Token(TokenType::UNKNOWN, "Unknown
character '" + std::string(1, bad) + "'", errLine,
errCol);
        }

        if (result.type != TokenType::COMMENT && result.type
!= TokenType::UNKNOWN) {
            prevTokenType = result.type;
        }

        return result;
    }
}

```

4. Fungsi main()

Merupakan *entry point* utama dari *program life cycle*. Fungsi ini mengelola inisialisasi *input file stream*, menginisiasi objek *Lexer*, dan mengeksekusi *loop* berkelanjutan yang secara konstan memanggil `getNextToken()`. *Loop* ini akan terus berputar hingga mesin DFA mengembalikan *token* `END_OF_FILE`. Selain itu, fungsi ini juga bertugas memformat hasil objek *token* menjadi teks untuk dicetak ke dalam `output.txt`.

```

int main(int argc, char* argv[]) {
    std::string baseDir = "test/milestone-1/";

    std::string inputFile = (argc > 1) ? (baseDir +
argv[1]) : (baseDir + "test1.txt");
    std::string outputFile = (argc > 2) ? (baseDir +
argv[2]) : (baseDir + "output.txt");

    std::ofstream out(outputFile);
    if (!out.is_open()) {

```

```

        std::cerr << "Error: Cannot open output file '" <<
outputFile << "'\n";
        return 1;
    }

    try {
        Lexer lexer(inputFile);

        while (true) {
            Token tok = lexer.getNextToken();

            if (tok.type == TokenType::UNKNOWN) {
                std::string errMsg = "Lexical Error at
Line " + std::to_string(tok.line) +
                                ", Col " +
std::to_string(tok.column) + ": " + tok.value;
                out << errMsg << "\n";
                std::cerr << errMsg << "\n";
            }
            else {
                out << tok.toString() << "\n";
            }

            if (tok.type == TokenType::END_OF_FILE) break;
        }

    } catch (const std::runtime_error& e) {
        std::cerr << e.what() << "\n";
        out << e.what() << "\n";
        return 1;
    }

    std::cout << "Tokenization complete. Output written to
'" << outputFile << "'.\n";
    return 0;
}

```

2.2.4. Justifikasi Implementasi

Implementasi *lexical analyzer* untuk bahasa Arion ini dibangun secara spesifik menggunakan bahasa pemrograman C++ standar GNU. Pemilihan bahasa C++ didasarkan pada keunggulannya dalam memberikan *low-level control* terhadap manajemen memori dan performa eksekusi komputasi yang sangat cepat, yang mana kedua hal ini merupakan aspek krusial dalam pemrosesan tahapan *compile*. Sesuai dengan spesifikasi tugas yang mensyaratkan pemahaman fundamental, pembuatan DFA dilakukan secara manual tanpa mengandalkan *auto-generator* seperti *Regex* ataupun *Flex*. Pembuatan secara manual ini memastikan bahwa mesin pembaca beroperasi murni berdasarkan transisi karakter, yang sangat berhubungan dengan cara kerja *lexer* di tingkat dasar.

Dari perspektif arsitektur perangkat lunak, program menggunakan paradigma OOP yang bersifat modular. Logika DFA tidak digabungkan dalam satu berkas yang monolitik atau digerakkan oleh konfigurasi eksternal, melainkan dipisahkan ke dalam kelas utama *dispatcher* dan beberapa berkas modul pembantu yang terisolasi (seperti pendeteksi literal, *keyword*, operator, dan komentar). Pemisahan tanggung jawab ini digunakan untuk mempermudah kolaborasi kode secara paralel dalam tim, meminimalisasi konflik saat menggabungkan kode, mempercepat proses *debugging*, serta mempermudah proses pengembangan program menuju *Syntax Analysis* pada *milestone* selanjutnya.

Penggunaan model DFA itu sendiri dipertahankan sebagai inti logika karena kemampuannya memproses input karakter demi karakter secara deterministik dan menggunakan metode *longest match*, sehingga memastikan setiap simbol dipetakan ke *state* berikutnya dengan jelas. Dalam mewujudkan transisi *state* yang kompleks, seperti membaca operator multi-karakter yang memerlukan *lookahead*, program tidak menggunakan implementasi *peek()*, melainkan menggunakan implementasi *method retract()*. Metode ini digunakan karena ia bertindak secara sama dengan operasi *retract* (yang ditandai dengan simbol bintang *) pada konsep DFA teoretis, di mana mesin dapat membatalkan dan mengembalikan karakter terakhir yang tidak relevan ke dalam *input stream* tanpa mengorbankan adanya perubahan data maupun performa pembacaan.

BAB 3

PENGUJIAN

3.1. Test Case 1

input1.txt
<pre> program Contoh; var x : integer; begin x:=10+5; end. </pre>

output1.txt
<pre> 1 PROGRAMSY 2 IDENT(Contoh) 3 SEMICOLON 4 VARSY 5 IDENT(x) 6 COLON 7 IDENT(integer) 8 SEMICOLON 9 BEGINSY 10 IDENT(x) 11 BECOMES 12 INTCON(10) 13 PLUS 14 INTCON(5) 15 SEMICOLON 16 STRING(' ') 17 ENDSY 18 PERIOD 19 END_OF_FILE </pre>

3.2. Test Case 2

input2.txt
<pre> program Hello-apa; my21var </pre>

```
    a, b: integer;  
begin  
    c:=3+2*5;  
    a := 5 ≤ 9;  
    b := 2..3;  
    writeln('Result = ', b*);  
    d := 'v';  
end.if
```

output2.txt


```
1  PROGRAMSY
2  IDENT>Hello)
3  MINUS
4  IDENT(apa)
5  SEMICOLON
6  IDENT(my21var)
7  IDENT(a)
8  COMMA
9  IDENT(b)
10 COLON
11 IDENT(integer)
12 SEMICOLON
13 BEGINSY
14 IDENT(c)
15 BECOMES
16 INTCON(3)
17 PLUS
18 INTCON(2)
19 TIMES
20 INTCON(5)
21 SEMICOLON
22 IDENT(a)
23 BECOMES
24 INTCON(5)
25 LEQ
26 INTCON(9)
27 SEMICOLON
28 IDENT(b)
29 BECOMES
30 INTCON(2)
31 PERIOD
32 PERIOD
33 INTCON(3)
34 SEMICOLON
35 IDENT(writeln)
36 LPARENT
37 STRING(Result = )
38 COMMA
39 IDENT(b)
40 TIMES
41 RPARENT
42 SEMICOLON
43 IDENT(d)
44 BECOMES
45 CHARCON(v)
46 SEMICOLON
47 ENDSY
48 PERIOD
49 IFSY
50 END_OF_FILE
```

3.3. Test Case 3

input3.txt
<pre>{ top-level comment with tokens: a := 1; .. 2 } (* multiline comment with 0..9 and writeln('x') *) begin x := 10; end.</pre>

output3.txt
<pre>1 COMMENT 2 COMMENT 3 BEGINSY 4 IDENT(x) 5 BECOMES 6 INTCON(10) 7 SEMICOLON 8 ENDSY 9 PERIOD 10 END_OF_FILE</pre>

3.4. Test Case 4

input4.txt
<pre>begin a := -67; b := +67; { unterminated comment... b := 7; s := 'missing end</pre>

output4.txt

```
1  BEGINSY
2  IDENT(a)
3  BECOMES
4  INTCON(-67)
5  SEMICOLON
6  IDENT(b)
7  BECOMES
8  PLUS
9  INTCON(67)
10 SEMICOLON
11 Lexical Error at Line 4, Col 5: Unterminated comment '{'
12 IDENT(b)
13 BECOMES
14 INTCON(7)
15 SEMICOLON
16 IDENT(s)
17 BECOMES
18 Lexical Error at Line 6, Col 10: Unterminated string
19 END_OF_FILE
```

3.5. Test Case 5

input5.txt

```
begin
  if a ≤ b then c:=1;
  if x◇0 then y:=2;
  while i≤10 do i:=i+1;

  writeln('It''s ''weird'', 'ok');
  ch := 'a';
  c2 := '';
end.
```

output5.txt

```
1  BEGINSY
2  IFSY
3  IDENT(a)
4  LEQ
5  IDENT(b)
6  THENSY
7  IDENT(c)
8  BECOMES
9  INTCON(1)
10 SEMICOLON
11 IFSY
12 IDENT(x)
13 NEQ
14 INTCON(0)
15 THENSY
16 IDENT(y)
17 BECOMES
18 INTCON(2)
19 SEMICOLON
20 WHILESY
21 IDENT(i)
22 LEQ
23 INTCON(10)
24 DOSY
25 IDENT(i)
26 BECOMES
27 IDENT(i)
28 PLUS
29 INTCON(1)
30 SEMICOLON
31 IDENT(writeln)
32 LPARENT
33 STRING(It's 'weird')
34 COMMA
35 STRING(ok)
36 RPARENT
37 SEMICOLON
38 IDENT(ch)
39 BECOMES
40 CHARCON(a)
41 SEMICOLON
42 IDENT(c2)
43 BECOMES
44 CHARCON(')
45 SEMICOLON
46 ENDSY
47 PERIOD
48 END_OF_FILE
```

BAB 4

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Dengan berpatokan pada dokumen [Spesifikasi Milestone 1 - Tubes IF2224 TBFO](#), telah berhasil diimplementasikan sebuah *Lexical Analyzer* berbasis *Deterministic Finite Automata* (DFA) untuk bahasa Arion sesuai dengan spesifikasi yang dijabarkan pada dokumen tersebut. *Lexer* berfungsi untuk mengubah kode Arion (dalam format .txt) yang masih berupa *character stream* menjadi rangkaian *token* yang memiliki makna (dalam format .txt), yang nantinya akan diproses lebih lanjut pada tahapan *parser*. *Tokenization* dilakukan dengan membaca setiap karakter yang ada pada kode program secara berurutan dan memetakan pola karakter tersebut ke dalam tabel *token* tertentu berdasarkan aturan transisi yang dijabarkan pada diagram DFA.

Untuk menjaga struktur kode tetap terorganisir dan mempermudah *maintainability* menuju tahapan *compile* selanjutnya, implementasi C++ tersebut dibangun secara modular menggunakan pendekatan Object-Oriented Programming. Program dirancang dengan modul pengarah utama yang mendistribusikan aliran karakter secara efisien ke berbagai fungsi sub-state DFA yang lebih spesifik. Modul-modul ini berhasil menangani pengklasifikasian 52 jenis *token* dengan baik, mulai dari pendeteksian literal (angka, karakter, dan teks string beserta *escape character* di dalamnya), pencocokan *keyword* serta *identifier* melalui *Lookup Table* yang bersifat *case-insensitive*, hingga pemrosesan operator simbolik menggunakan logika *lookahead* yang diwujudkan melalui metode *peek*. Selain itu, program juga telah tahan dalam menangani *edge-case*, seperti mengabaikan *whitespace* yang tidak relevan, membuang blok komentar, serta memunculkan pesan *lexical error* apabila mendeteksi *state* yang menggantung akibat string atau komentar yang gagal ditutup. Pada tahap akhir, seluruh *token* valid yang berhasil diidentifikasi akan distandarisasi ke dalam format *lowercase*, sehingga keluaran yang dicetak dipastikan siap untuk diteruskan ke tahapan *Syntax Analysis* pada milestone berikutnya.

4.2 Saran

Beberapa spesifikasi yang tertera pada dokumen spesifikasi tidak dijelaskan secara rinci, sehingga banyak pertanyaan pada sesi QNA yang sebenarnya bisa dihindari dengan penjelasan yang lebih rinci pada dokumen spesifikasi. Selain itu juga, terdapat inkonsistensi yang muncul, contohnya pada format output untuk “string(‘Result = ‘)” yang berbeda antara dokumen spesifikasi dengan hasil sesi QNA, dan masih ada beberapa kasus lainnya. Semoga beberapa saran ini bisa dijadikan pertimbangan untuk milestone-milestone selanjutnya, sehingga para mahasiswa yang mengambil mata kuliah ini di semester ini bisa mengerjakan tugas besar dengan lebih lancar.

LAMPIRAN

Tautan repository Github:

<https://github.com/Rizelbit/SBT-Tubes-IF2224-2026>

Tautan Diagram DFA:

<https://drive.google.com/file/d/1JQkafdOFwvhX3yxJjynyNv-1ERKnLzq8/view?usp=sharing>

Tabel Pembagian Tugas

No.	NIM	Persentase Pembagian Tugas (%)
1.	13524032	25
2.	13524036	25
3.	13524056	25
4.	13524102	25

REFERENSI

“Introduction to Automata Theory, Languages, and Computation 3rd Edition”

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

“Compilers - Principles, Techniques, and Tools (2006)”

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)